MACHINE LEARNING IN PHYSICS
TRANSFORMERS

HARRISON B. PROSPER

PHY6937 / PHY4936

Outline

- Sequence to Sequence (Seq2Seq) Models
- Transformers
- Summary

Sequence to Sequence Models 1/9

A sequence to sequence (seq2seq) model maps one sequence of tokens to another. A word, part of a word, or a symbol are examples of tokens.

Examples:

1. the white house $\Longrightarrow$ la maison blanche
2. $\cosh^3(ax) + \tanh(bx) \Longrightarrow 1 + bx + \frac{3a^2x^2}{2} - \frac{b^3x^3}{3} + \cdots$

Tokens: the, white, house, la, maison, blanche, cosh, tanh, *x*, *a*, *b*, … etc.

Sequence to Sequence Models 2/9

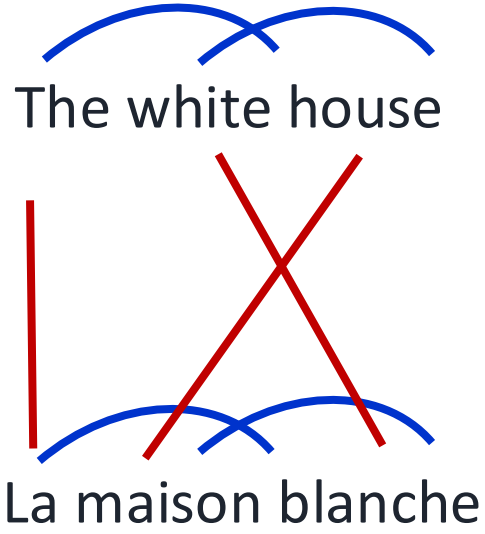
Consider what a model needs to do to solve the translation task:

$$x \Longrightarrow y$$

The white house $\Longrightarrow$ La maison blanche

1. The model must pay *attention* to the order of "The", "white", and "house" and "La", "maison", and "blanche".

2. It must also pay *attention* to the association between "The" and "La", "white" and "blanche", and "house" and "maison".



Self-Attention

Attention

Seq2Seq Models: Steps 3/9

A seq2seq model generally implements the following steps.

1. Tokenization: splitting a sequence into tokens.
2. Coding: assigning a unique integer to each unique token.
3. Embedding: representing each token by a vector.
4. Analysis: analyzing the input (prompt) sequence.
5. Synthesis: constructing the output (target) sequence.

Seq2Seq: Tokenization 4/9

The prompt $\cosh^3(ax) + \tanh(bx)$ can be split into the tokens: cosh, 3, (, *a*, *x*,), +, tanh, *b*.

Prompt

$$\cosh^3(ax) + \tanh(bx)$$

Target

$$1 + bx - \frac{b^3x^3}{3} + \frac{2b^5x^5}{15} + \frac{3a^2x^2}{2} + \frac{7a^4x^4}{8} + O\left(x^6\right)$$

The target can also be split into tokens.

Seq2Seq: Tokenization 5/9

The set of unique tokens is called a vocabulary. Here is the vocabulary of the tiny language model (TinyLM) we shall later explore in detail:

```
build vocabulary
{'<pad>': 0, '<sos>': 1, '<eos>': 2, '<sep>': 3, ' ': 4, '(': 5,
')': 6, '*': 7, '**': 8, '+': 9, '-': 10, '/': 11, '0': 12, '1': 13
, '2': 14, '3': 15, '4': 16, '5': 17, '6': 18, '7': 19, '8': 20, '9
': 21, 'O(x**6)': 22, 'a': 23, 'b': 24, 'c': 25, 'cos': 26, 'cosh':
27, 'd': 28, 'exp': 29, 'f': 30, 'g': 31, 'h': 32, 'm': 33, 'n': 34
, 'sin': 35, 'sinh': 36, 'tan': 37, 'tanh': 38, 'x': 39}
```

Large-language models (LLMs) like qwen3:8b, which has 8 billion parameters, have vocabularies of order 100,000 tokens.

Seq2Seq: Coding/Padding 6/9

Since machines work with numbers it is necessary to map every token to a number (e.g., an integer). Here is the sequence of codes for our example prompt and target.

```
Sequence
[ 1 27  5 23  7 39  6  8 15  9 38  5 24  7 39  6  3 13  9 24  7 39  9 15
  7 23  8 14  7 39  8 14 11 14 10 24  8 15  7 39  8 15 11 15  9 19  7 23
  8 16  7 39  8 16 11 20  9 14  7 24  8 17  7 39  8 17 11 13 17  9 22  2
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0]
```

In our (tiny) vocabulary, 3 is the code for a separator (SEP) token between the prompt and the target, while codes 1 and 2 denote the start-of-sequence (SOS) and end-of-sequence (EOS), respectively, and 0 are pad (PAD) tokens.

Seq2Seq: Coding/Padding 7/9

In current LLMs, the input sequence of tokens must fit within the context window, which is the maximum number of input tokens that can be processed by the LLM. For qwen3:8b the context window is of $\mathcal{O}(10^6)$ tokens. For our tiny language model, the context window is 128 tokens long!

```
Sequence
[ 1 27  5 23  7 39  6  8 15  9 38  5 24  7 39  6  3 13  9 24  7 39  9 15
  7 23  8 14  7 39  8 14 11 14 10 24  8 15  7 39  8 15 11 15  9 19  7 23
  8 16  7 39  8 16 11 20  9 14  7 24  8 17  7 39  8 17 11 13 17  9 22  2
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0]
```

Seq2Seq: Embedding 8/9

An embedding space, a key idea in many machine learning models, is a vector space in which data elements, here tokens, are represented as vectors.

Example 1: In our CAE implementation (Labs/04.CNN), each (3,96,96) image is mapped to a (16,6,6) image space.

Example 2: In our TinyLM tutorial (Labs/10.Transformer), each token and its ordinal position is mapped to a 72-dimensional vector space.

Seq2Seq: Analysis/Synthesis 9/9

- Seq2seq models, like recurrent neural networks (RNN), long short-term memories (LSTM), and transformer neural networks (TNN) – the subject of today's lecture – analyze and synthesize sequences in different ways.

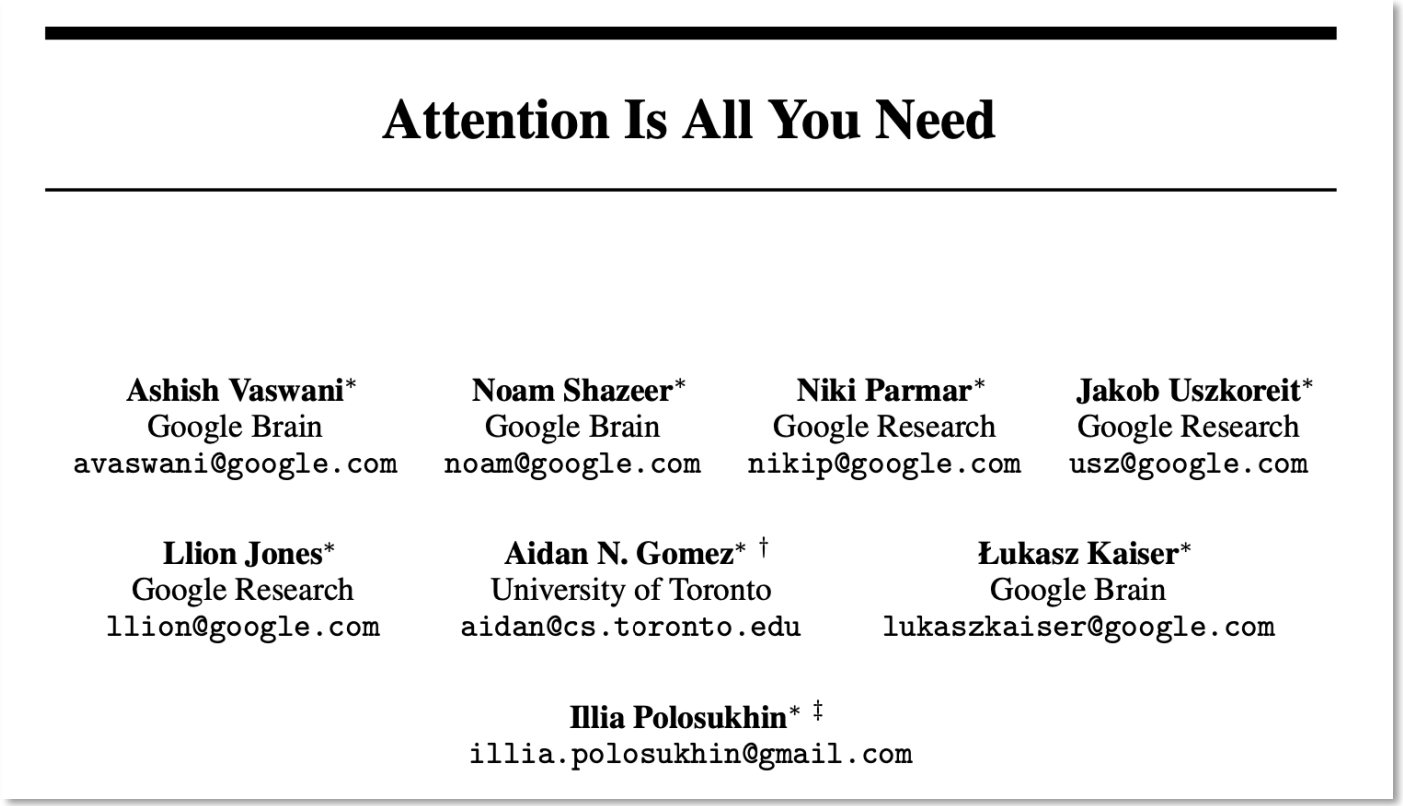
- Moreover, some classes of seq2seq models (e.g., RNN, LSTM) analyze sequences one token at a time.

- Transformers were a breakthrough, in part, because tokens are analyzed in *parallel*.

TRANSFORMER
NEURAL NETWORKS

Attention Is All You Need

In 2017, in a seminal paper: https://arxiv.org/abs/1706.03762
Google researchers introduced a transformer neural network (TNN) with a highly successful analysis and synthesis method.



Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

The Transformer 1/3

- Current transformers are decoder models that map one sequence of tokens to another $x \Longrightarrow y$.

- During training, the input sequence (comprising a prompt x and the associated output sequence y) enters the decoder *in parallel*.

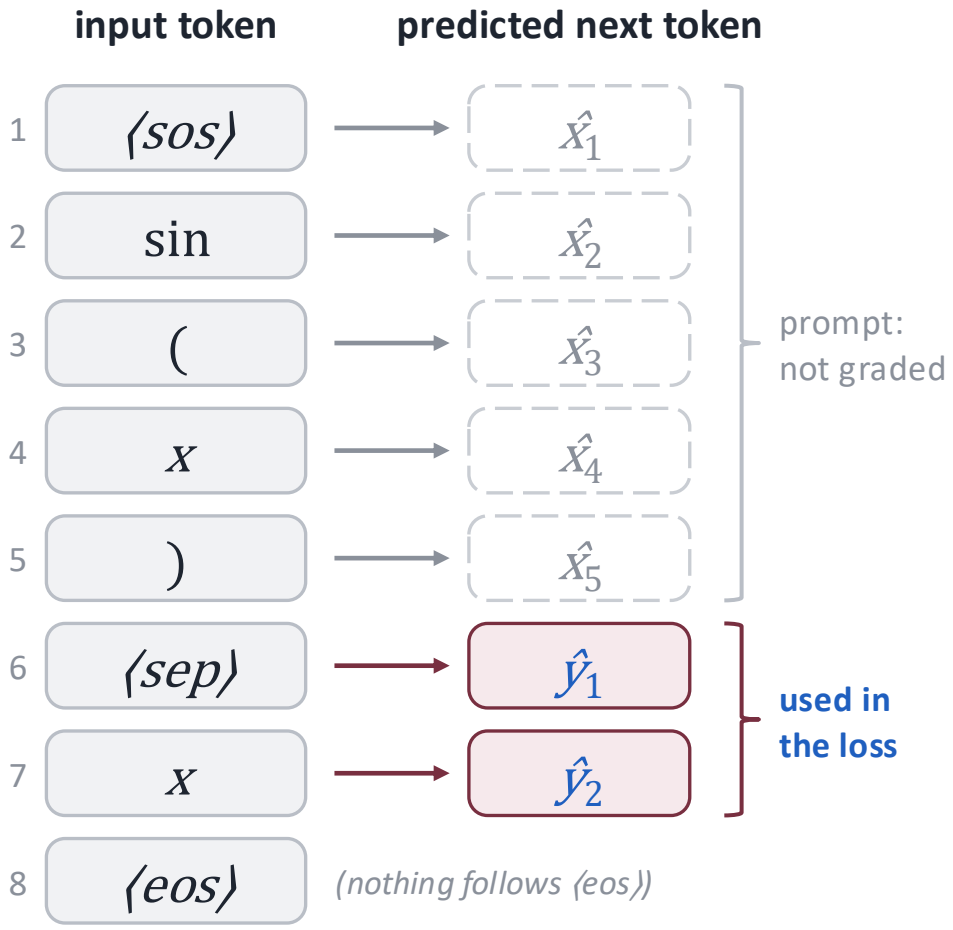
- The decoder computes probabilities which are used to predict, again in parallel, the next output token *for every sub-sequence of the input sequence*.

- The losses are computed, again in parallel, using only the tokens of the predicted output sequence and the true output tokens.

The Transformer 2/3

Example

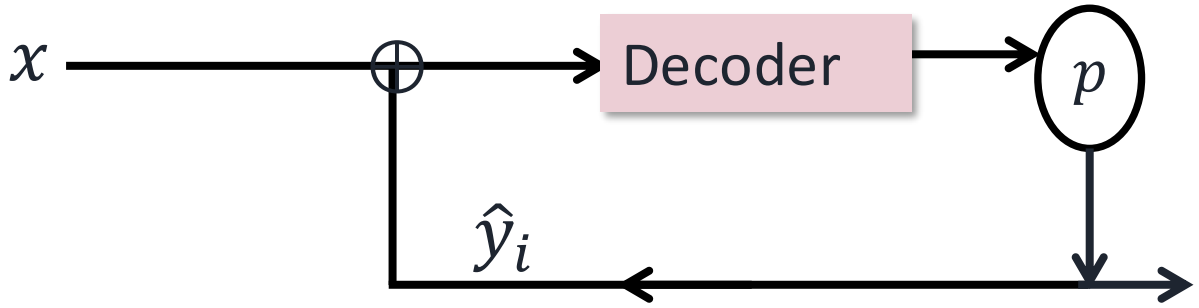
- Consider a transformer with an 8-token wide context window into which the following 8-token long input sequence is entered: $\langle sos \rangle, \sin, (, x,), \langle sep \rangle, x, \langle eos \rangle$
- During training, this sequence enters the model in parallel, and, for every sub-sequence, the model predicts the next token. Only the predictions $\hat{y}_1$ and $\hat{y}_2$ are used in the calculation of the losses.



The Transformer 3/3

A **trained model** is used *autoregressively*.

- The separator token, $\langle sep \rangle$, is appended to the prompt sequence x and the updated sequence x is passed, in parallel, to the decoder.
- The decoder predicts probabilities for the next token for *every* sub-sequence of x, but only the probabilities associated with the *longest* sub-sequence are used to select the next output token, which is appended to x and output sequence $\hat{y}$.
- The cycle repeats until the next token is either the end-of-sequence, $\langle eos \rangle$, or the token count limit is reached.

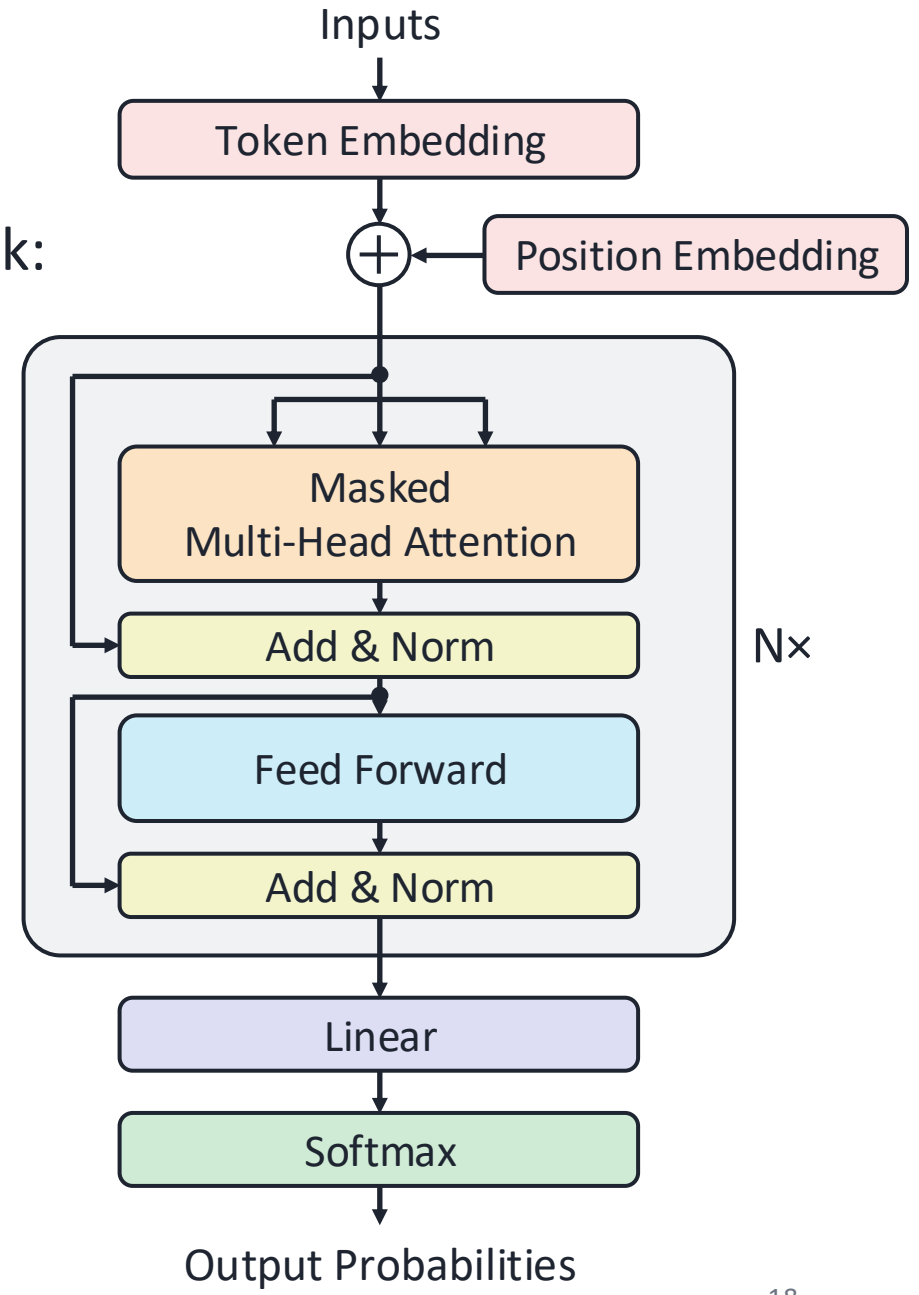


THE TINY LANGUAGE MODEL

The tiny language model (TinyLM)

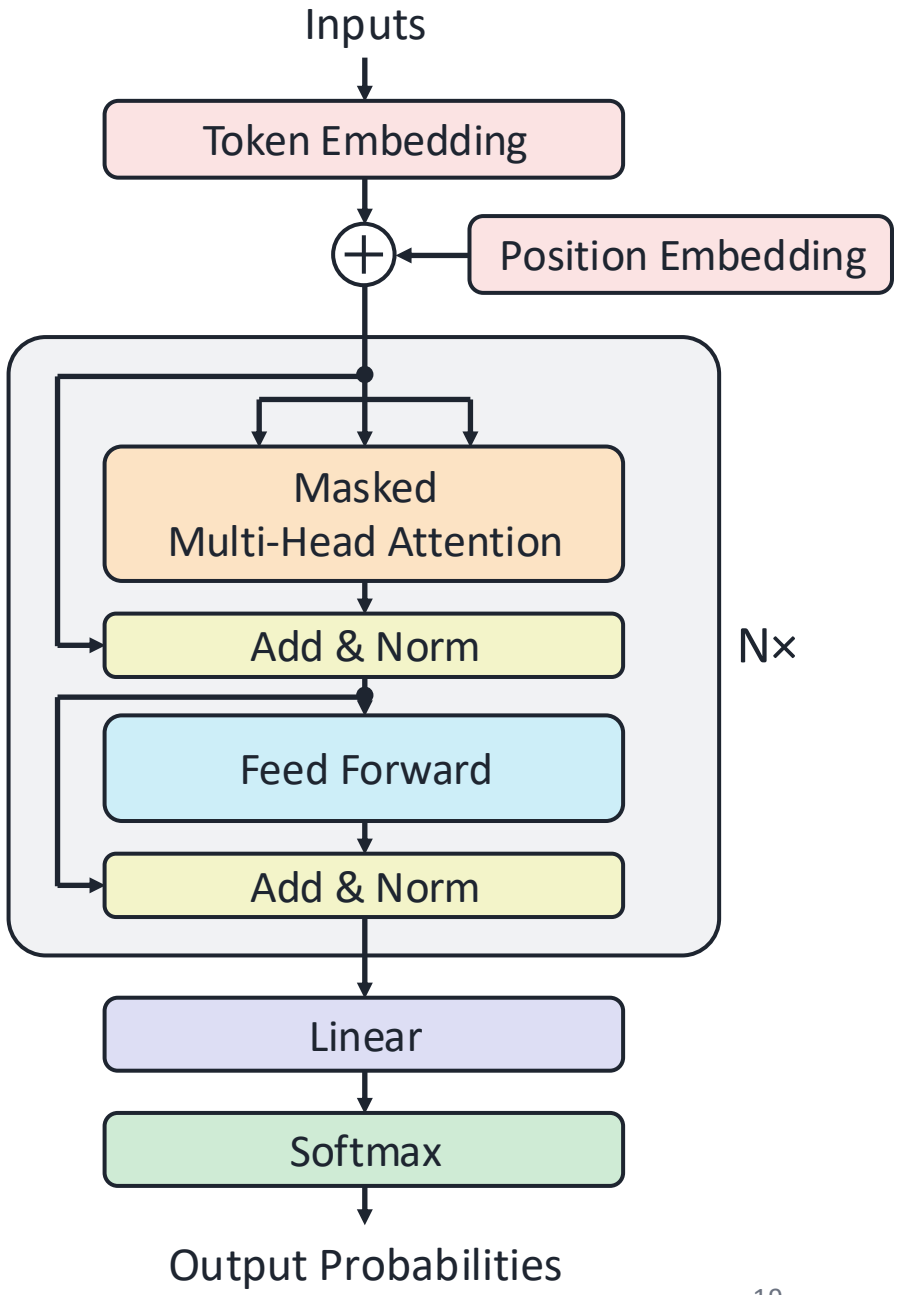
We shall build a model comprising three kinds of block:

- Embedding block
- Decoding blocks
- Probability block

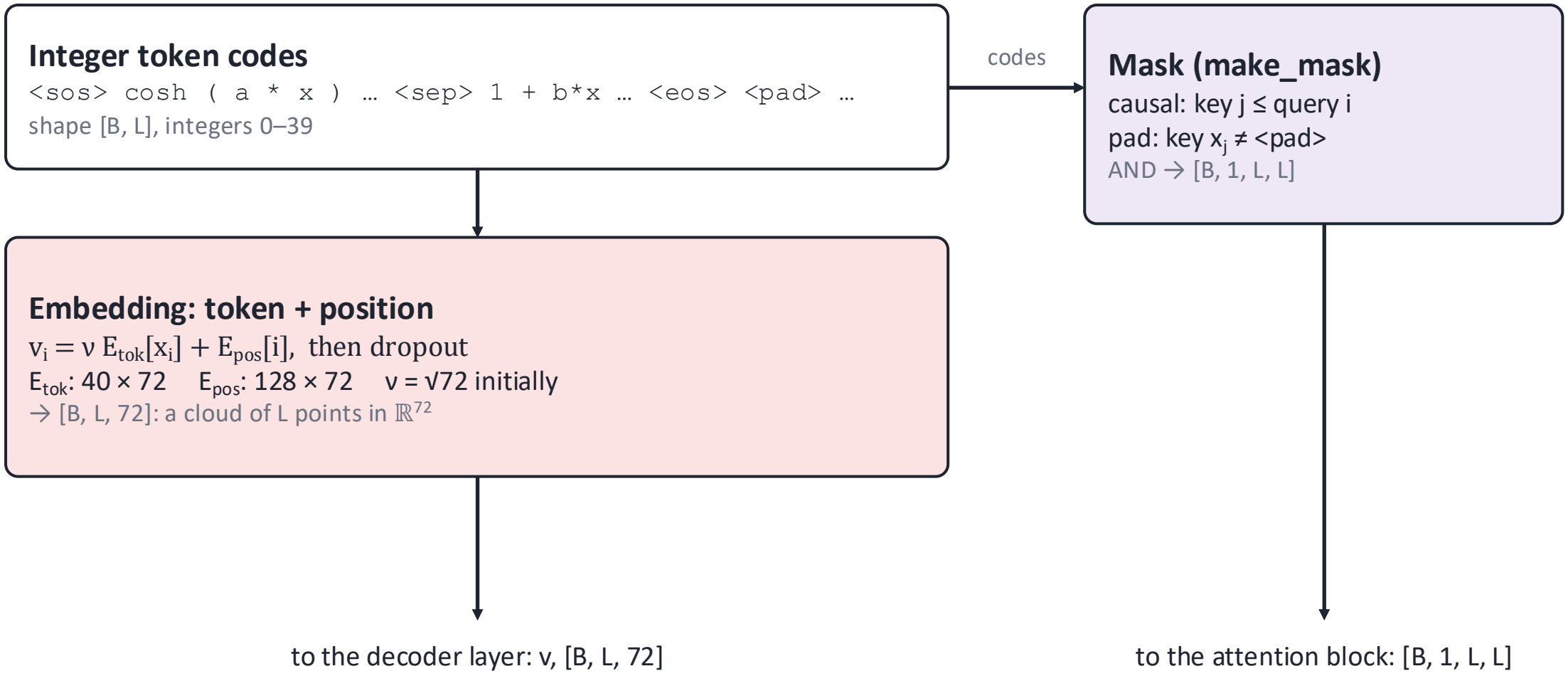


The tiny language model (TinyLM)

- Embedding block
- Decoding blocks
- Probability block

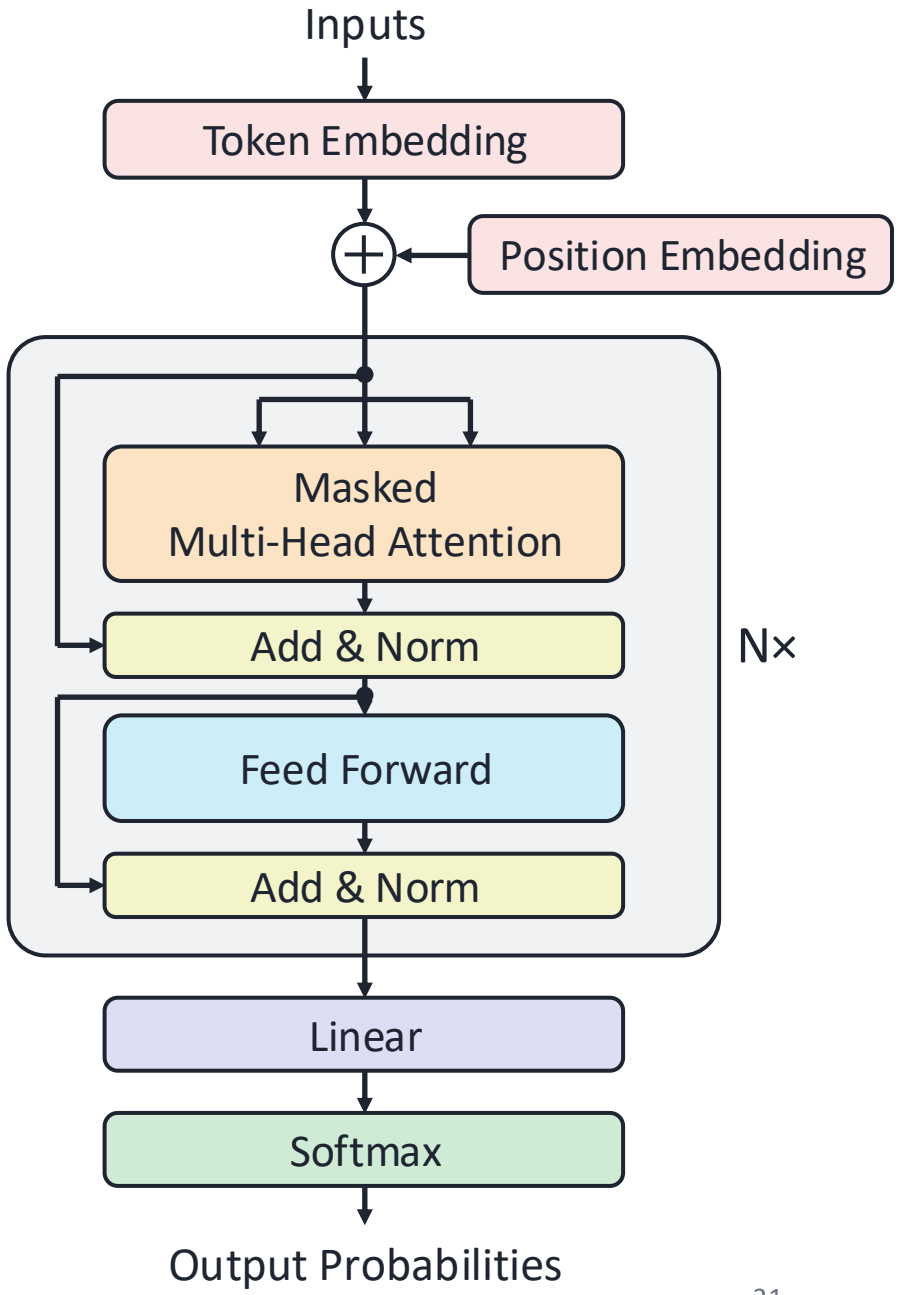


TinyLM in detail 1/3: Tokens to vectors and causal masking

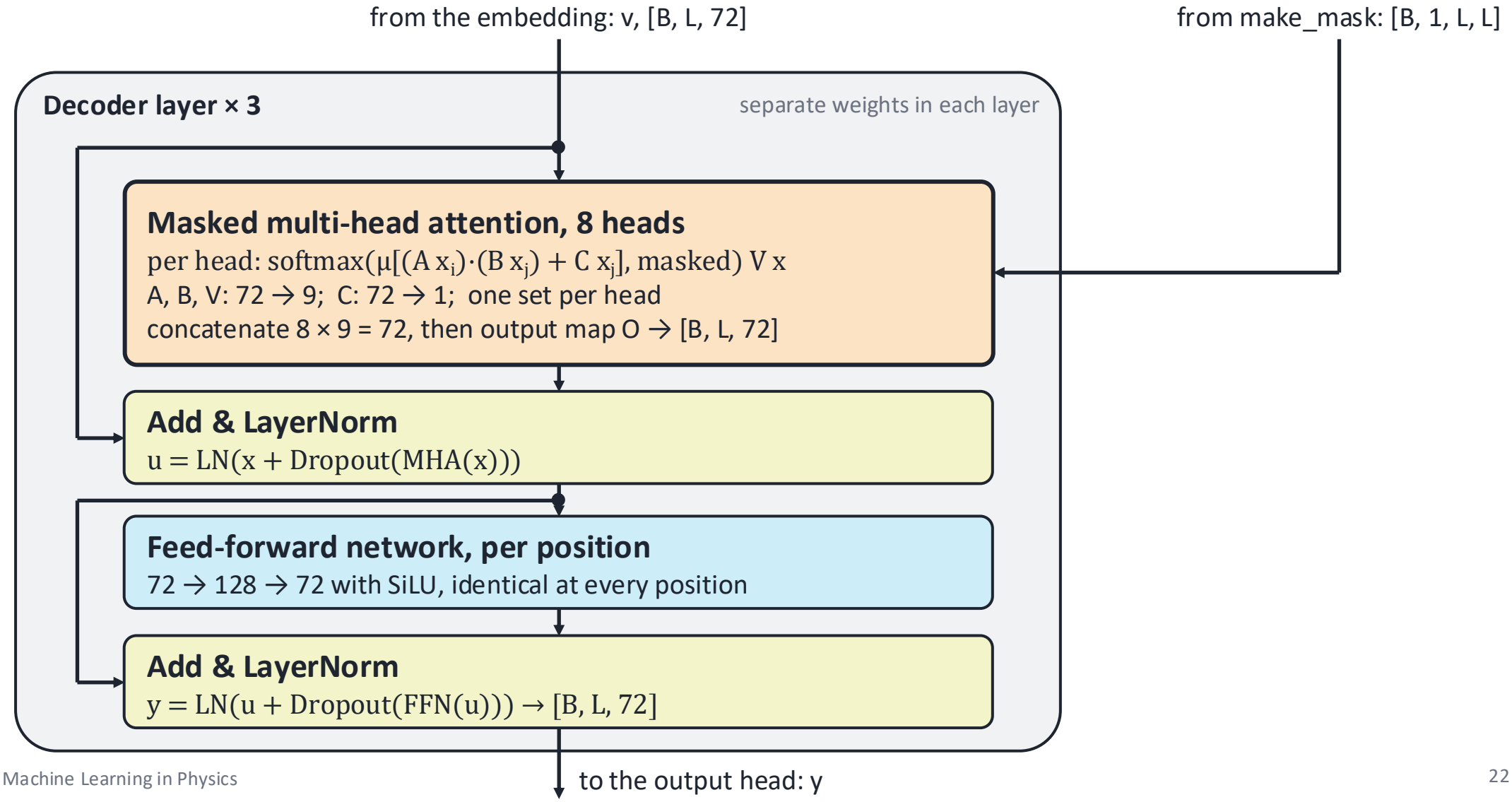


The tiny language model (TinyLM)

- Embedding block
- Decoding blocks
- Probability block



TinyLM in detail 2/3: The decoder layer, repeated 3 times



Attention 1/6

- Attention is a function defined on two sequences of tokens that measures the *degree of alignment* between any pair of tokens (or elements).

- Self-attention is attention with identical sequences. Models like Claude, Gemini, ChatGPT use self-attention.

Attention 2/6

At the heart of Google's breakthrough model is the following expression,

$$\text{Attention}(Q, K, V) = \text{softmax}(\mu Q K^T) V$$

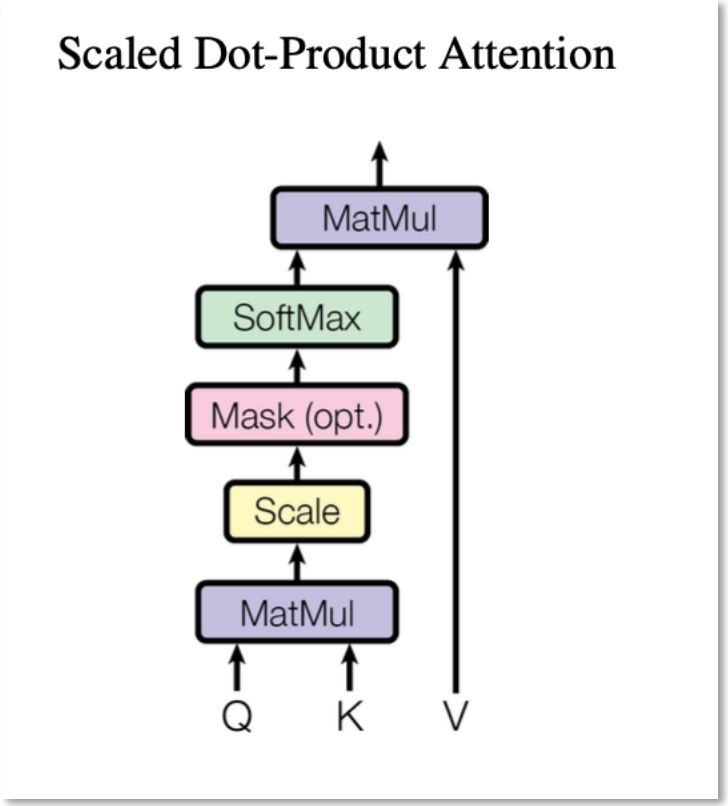
which captures some notion of a token "paying attention to" other tokens.

Q: query,

K: key,

V: value

These matrices are linear functions of the same input sequence x.



https://arxiv.org/abs/1706.03762

Attention 3/6: Token vectors

In what sense does

$$\text{Attention}(Q, K, V) = \text{softmax}(\mu Q K^T) V$$

capture the notion of a token "*paying attention to*" other tokens?

Attention 4/6: Dot products

- Consider a sequence of 2 tokens, each represented by a vector.

- The matrices can be viewed as a columns of vectors, each associated with a token,

$$Q = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}, K = \begin{bmatrix} k_1 \\ k_2 \end{bmatrix}, V = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$$

- The matrix (outer) product QK^T is the matrix of dot products

$$QK^T = \begin{bmatrix} q_1 \cdot k_1 & q_1 \cdot k_2 \\ q_2 \cdot k_1 & q_2 \cdot k_2 \end{bmatrix}$$

Attention 5/6: Scaled dot products

The matrix of *scaled* dot products, after **`softmax`**, is given by

$$\text{softmax}(\mu Q K^T) = \begin{bmatrix} \exp(\mu q_1 \cdot k_1)/a_1 & \exp(\mu q_1 \cdot k_2)/a_1 \\ \exp(\mu q_2 \cdot k_1)/a_2 & \exp(\mu q_2 \cdot k_2)/a_2 \end{bmatrix}$$

where a_1 and a_2 are normalization factors that ensure each row sums to one.

The above matrix of *normalized* weights is then multiplied by the matrix V. The upshot is that each token is associated with a weighted sum of token vectors, presumably, the ones to which a given token "*pays attention*".

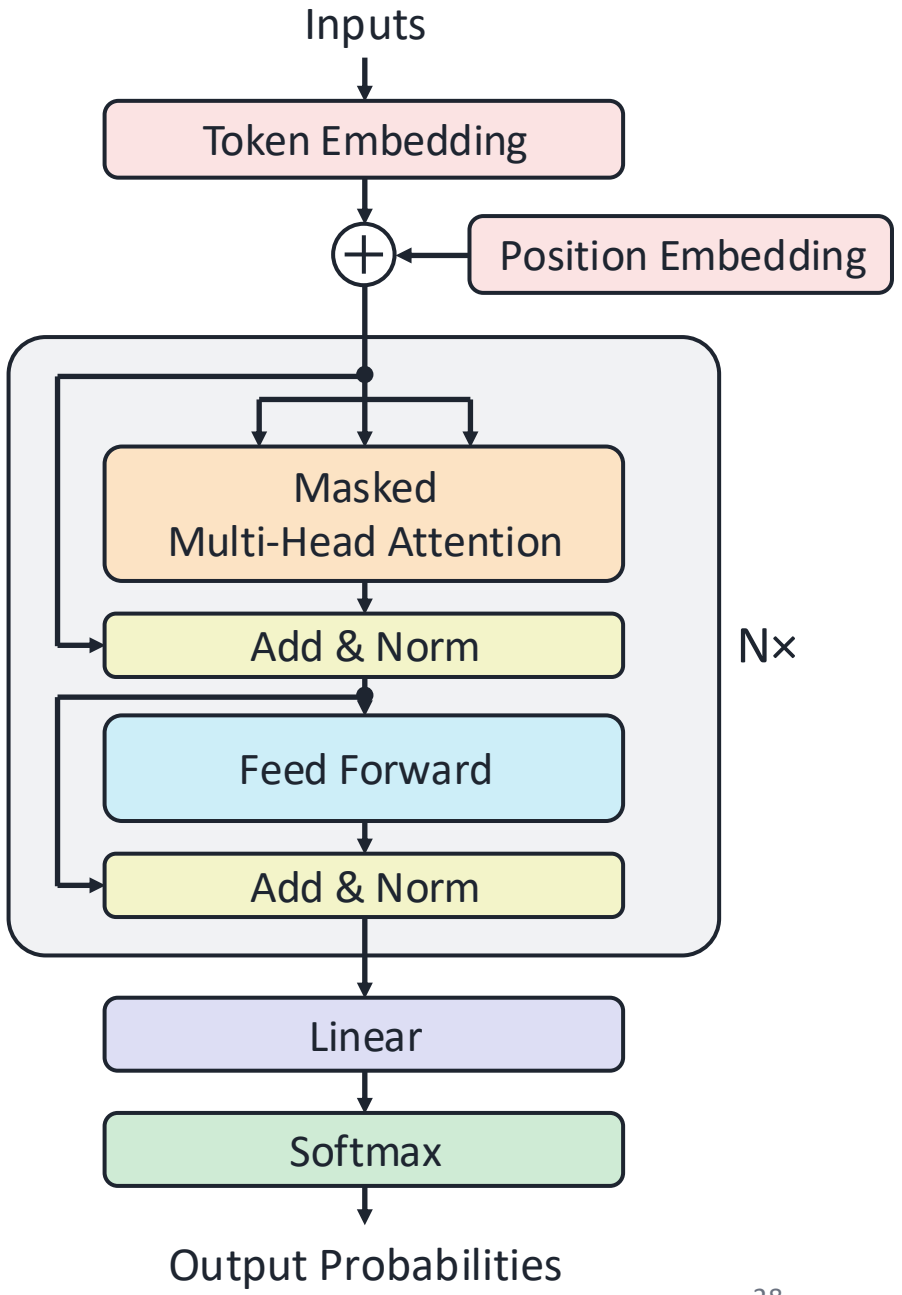
Attention 6/6: ABC parametrization

For self-attention, where x is a sequence of tokens

$$Q = xA_Q^T + b_Q,$$
$$K = xA_K^T + b_K, \text{ and}$$
$$V = xA_V^T + b_V,$$

the expression $\mathrm{softmax}(\mu Q K^T)V$ reduces to an expression of the form

$$\mathrm{Attention}(x) = \mathrm{softmax}(\mu[x\, A^T B\, x^T + C\, x^T])\, V(x)$$



Masking

- As noted, the attention mechanism compares every input token with every other token, which means we must deal with large $n \times n$ matrices.
- In the following slides, the vertical direction (the “query”) of these matrices corresponds to an input sequence.
- The horizontal (“key”) direction corresponds to the tokens to which a given token “pays attention”.
- A pad mask ensures that pad tokens are not used. A causal mask prevents an input token from “paying attention to” tokens that are ahead of it.

Masking

one attention head · 8 tokens

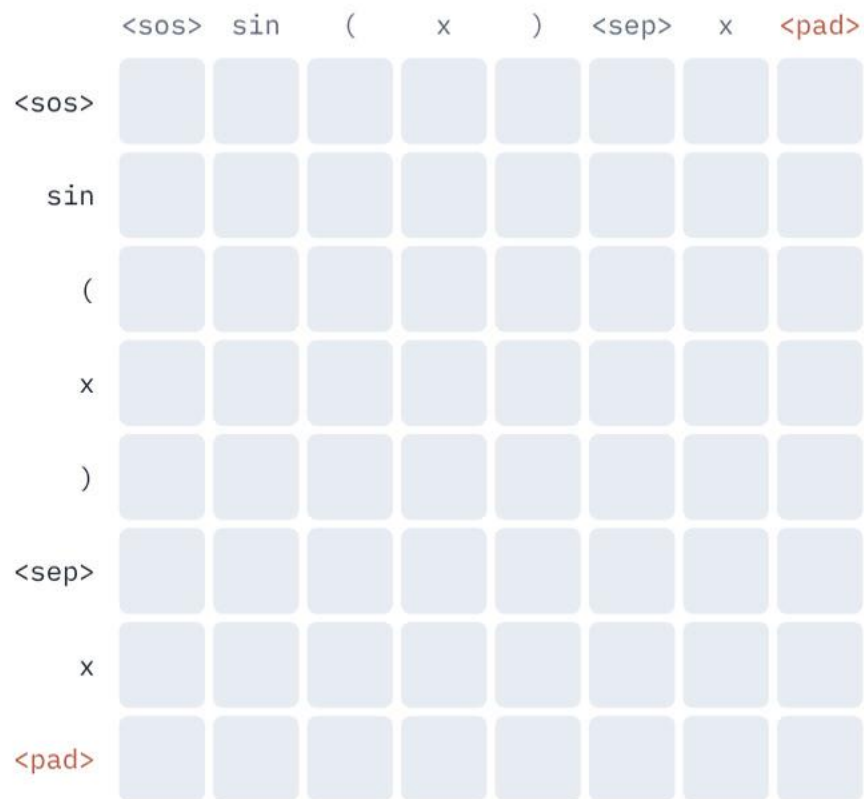
Masking

one attention head · 8 tokens

Pad mask



Causal mask = tril(ones)

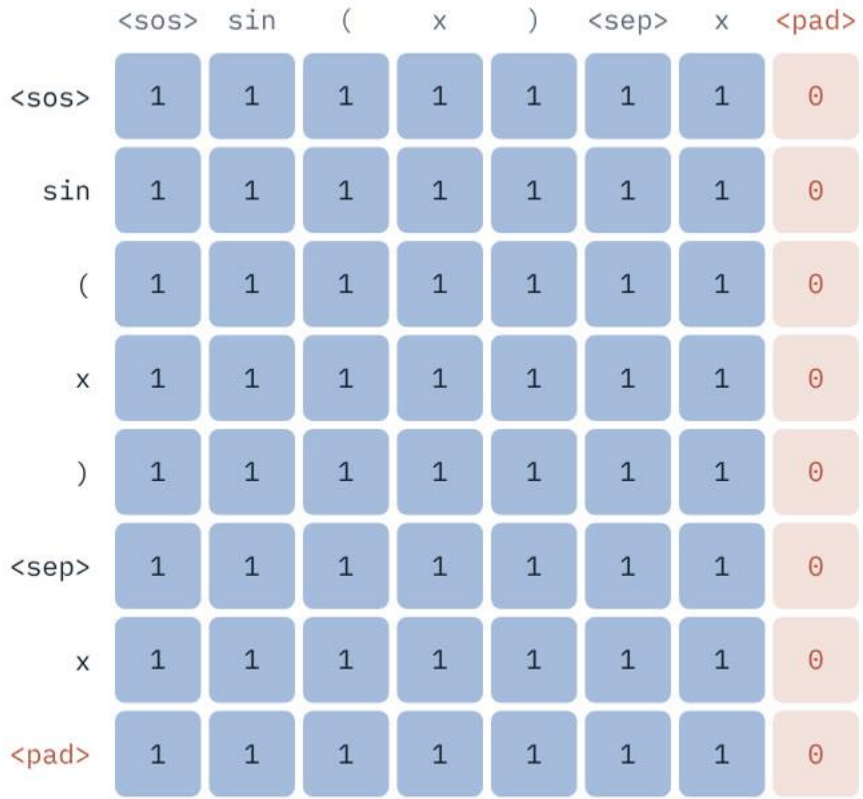


Two 8 × 8 masks over queries i (rows) and keys j (columns).

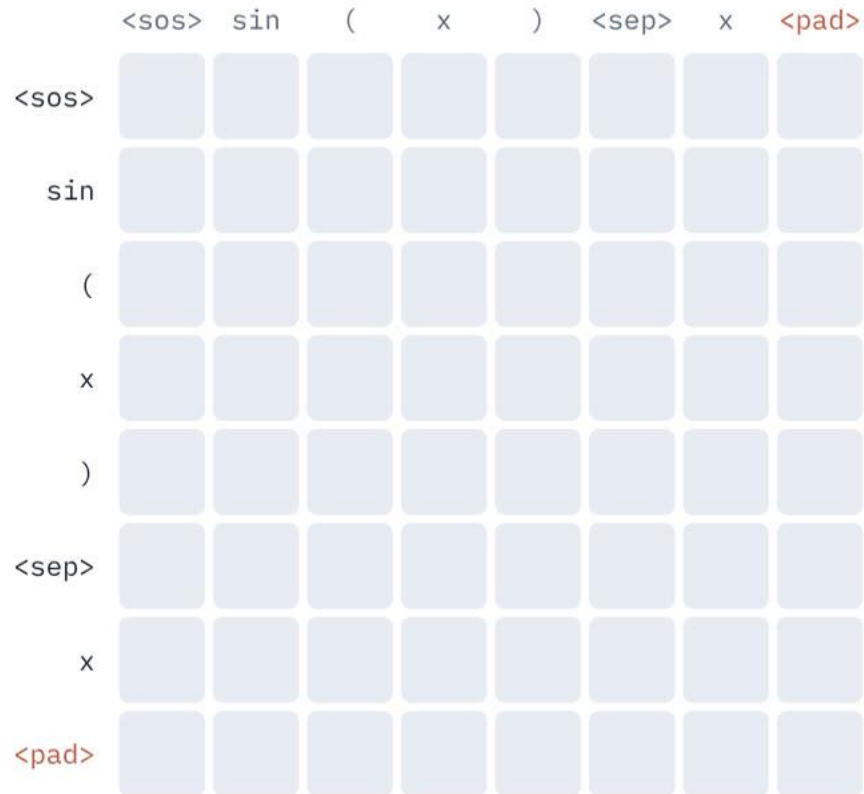
Masking

one attention head · 8 tokens

Pad mask



Causal mask = tril(ones)



Pad mask: 1 where key j is a real token, 0 where it is <pad>.

Masking

one attention head · 8 tokens

Pad mask

	<sos>	sin	(	x	)	<sep>	x	<pad>
<sos>	1	1	1	1	1	1	1	0
sin	1	1	1	1	1	1	1	0
(	1	1	1	1	1	1	1	0
x	1	1	1	1	1	1	1	0
)	1	1	1	1	1	1	1	0
<sep>	1	1	1	1	1	1	1	0
x	1	1	1	1	1	1	1	0
<pad>	1	1	1	1	1	1	1	0

Causal mask = tril(ones)

	<sos>	sin	(	x	)	<sep>	x	<pad>
<sos>	1	0	0	0	0	0	0	0
sin	1	1	0	0	0	0	0	0
(	1	1	1	0	0	0	0	0
x	1	1	1	1	0	0	0	0
)	1	1	1	1	1	0	0	0
<sep>	1	1	1	1	1	1	0	0
x	1	1	1	1	1	1	1	0
<pad>	1	1	1	1	1	1	1	1

Causal mask = tril(ones): row i may look only at keys 0…i.

Masking

one attention head · 8 tokens

Combined mask = pad AND causal

	<sos>	sin	(	x	)	<sep>	x	<pad>
<sos>	1	0	0	0	0	0	0	0
sin	1	1	0	0	0	0	0	0
(	1	1	1	0	0	0	0	0
x	1	1	1	1	0	0	0	0
)	1	1	1	1	1	0	0	0
<sep>	1	1	1	1	1	1	0	0
x	1	1	1	1	1	1	1	0
<pad>	1	1	1	1	1	1	1	0

Combine them with AND: an entry survives only if both masks are 1.

Masking

one attention head · 8 tokens

Scores S_{ij} (before the softmax)

	<sos>	sin	(	x	)	<sep>	x	<pad>
<sos>	2.1	1.2	-1.7	0.5	-1.1	-1.2	2.2	-0.2
sin	2.2	-0.1	0.9	0.1	-2.4	0.8	-0.1	-0.2
(	-0.5	0.6	2.6	-1.6	-0.1	-0.2	-1.8	1.9
x	-1.1	2.3	-0.1	0.7	0.9	-2.4	0.0	0.7
)	0.9	0.3	-0.2	2.3	-0.4	-1.0	0.5	-1.9
<sep>	0.1	-1.6	2.0	0.7	-0.7	1.8	-1.9	-0.9
x	-2.4	0.1	1.1	-0.7	2.0	-0.1	-1.1	0.7
<pad>	-1.3	0.7	-1.6	1.2	1.7	-1.1	1.0	-0.4

The head's scores before the softmax: $S_{ij} = \mu[(Ax_i)\cdot(Bx_j) + Cx_j]$.

Masking

one attention head · 8 tokens

Scores S_{ij} (before the softmax)

	<sos>	sin	(	x	)	<sep>	x	<pad>
<sos>	2.1	−∞	−∞	−∞	−∞	−∞	−∞	−∞
sin	2.2	-0.1	−∞	−∞	−∞	−∞	−∞	−∞
(	-0.5	0.6	2.6	−∞	−∞	−∞	−∞	−∞
x	-1.1	2.3	-0.1	0.7	−∞	−∞	−∞	−∞
)	0.9	0.3	-0.2	2.3	-0.4	−∞	−∞	−∞
<sep>	0.1	-1.6	2.0	0.7	-0.7	1.8	−∞	−∞
x	-2.4	0.1	1.1	-0.7	2.0	-0.1	-1.1	−∞
<pad>	-1.3	0.7	-1.6	1.2	1.7	-1.1	1.0	−∞

Combined mask

	<sos>	sin	(	x	)	<sep>	x	<pad>
<sos>	1	0	0	0	0	0	0	0
sin	1	1	0	0	0	0	0	0
(	1	1	1	0	0	0	0	0
x	1	1	1	1	0	0	0	0
)	1	1	1	1	1	0	0	0
<sep>	1	1	1	1	1	1	0	0
x	1	1	1	1	1	1	1	0
<pad>	1	1	1	1	1	1	1	0

Wherever the combined mask is 0, the score becomes -10^{10} (shown as −∞).

Attention as a weighted average

one head · query token k



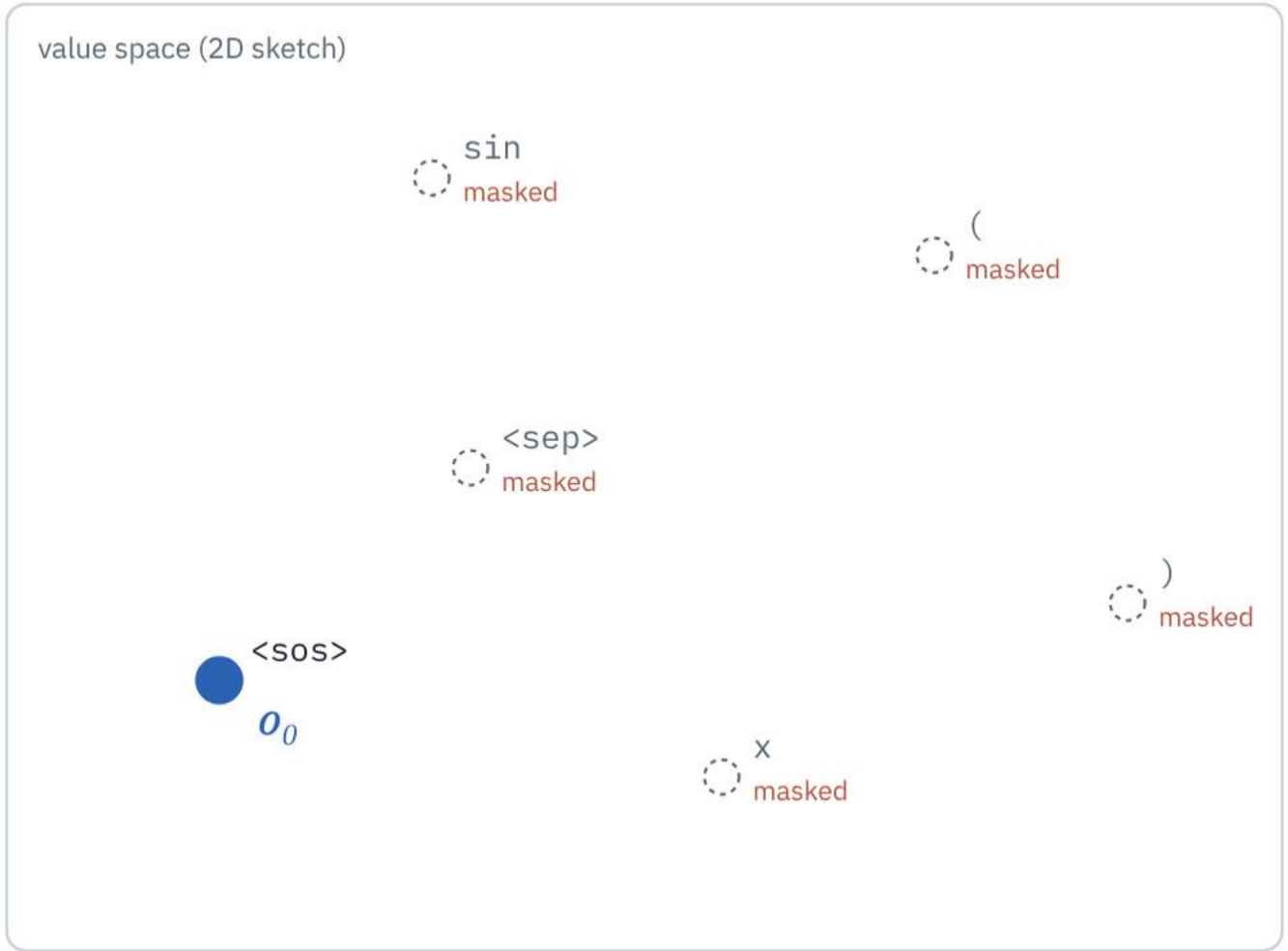
Weights w_{kj}

<sos>
sin
(
x
)
<sep>

$$o_k = \Sigma_{j \le k}\, w_{kj} \cdot V x_j$$

Attention as a weighted average

one head · query token k



Weights for query k = 0 (<sos>)

<sos>	1.00
sin	0 (masked)
(	0 (masked)
x	0 (masked)
)	0 (masked)
<sep>	0 (masked)

Σ = 1.00

$$o_k = \Sigma_{j \le k}\, w_{kj} \cdot V x_j$$

Attention as a weighted average

one head · query token k



Weights for query k = 1 (sin)

<sos>	1.00	
sin	0.00	
(	0 (masked)	
x	0 (masked)	
)	0 (masked)	
<sep>	0 (masked)	

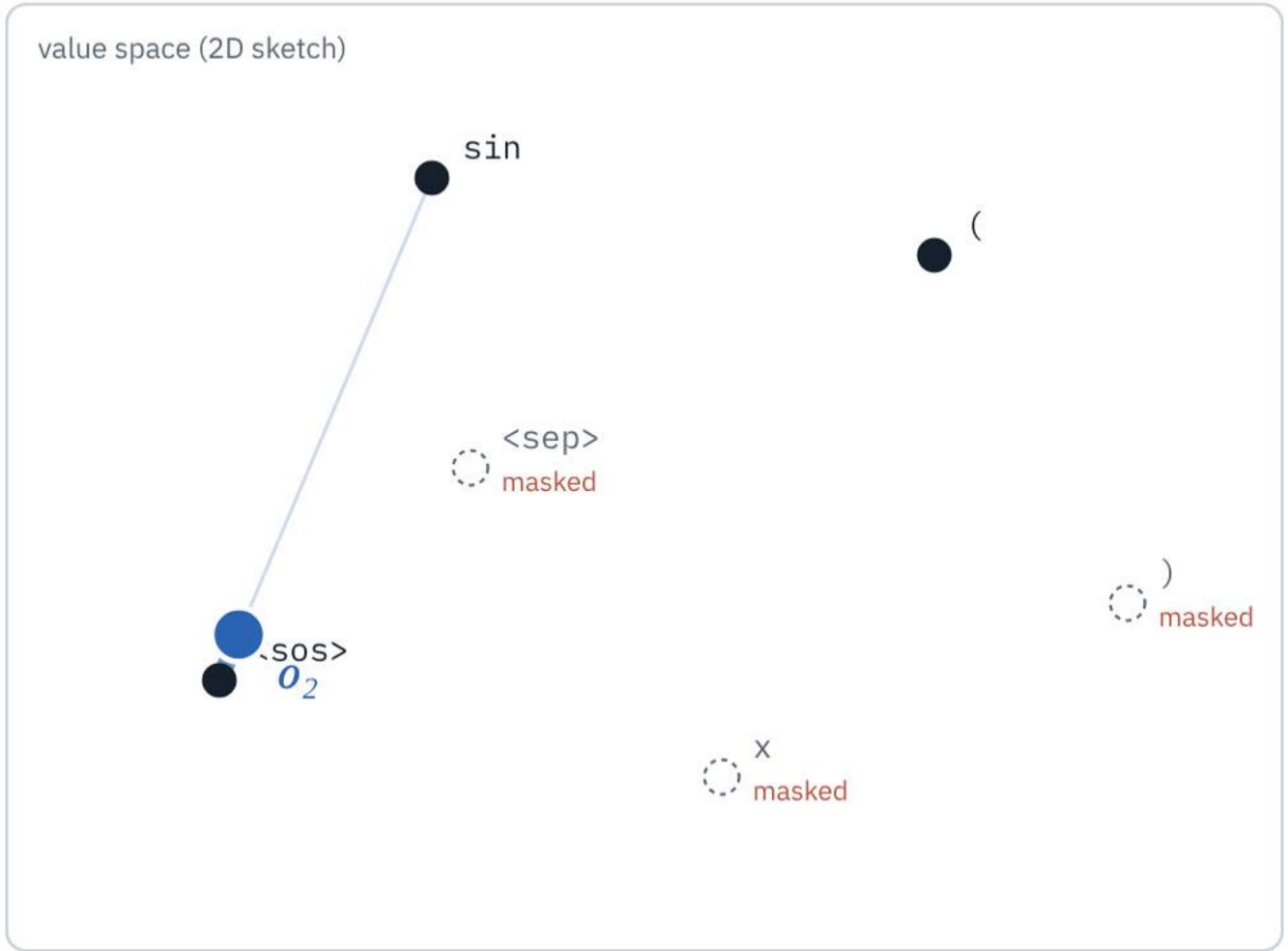
Σ = 1.00

$$o_k = \Sigma_{j \le k}\, w_{kj} \cdot V x_j$$

Query k = 1: the output is a weighted average of tokens 0…1; later tokens get weight 0.

Attention as a weighted average

one head · query token k



Weights for query k = 2 (()

<sos>	0.91	
sin	0.09	
(	0.00	
x	0 (masked)	
)	0 (masked)	
<sep>	0 (masked)	

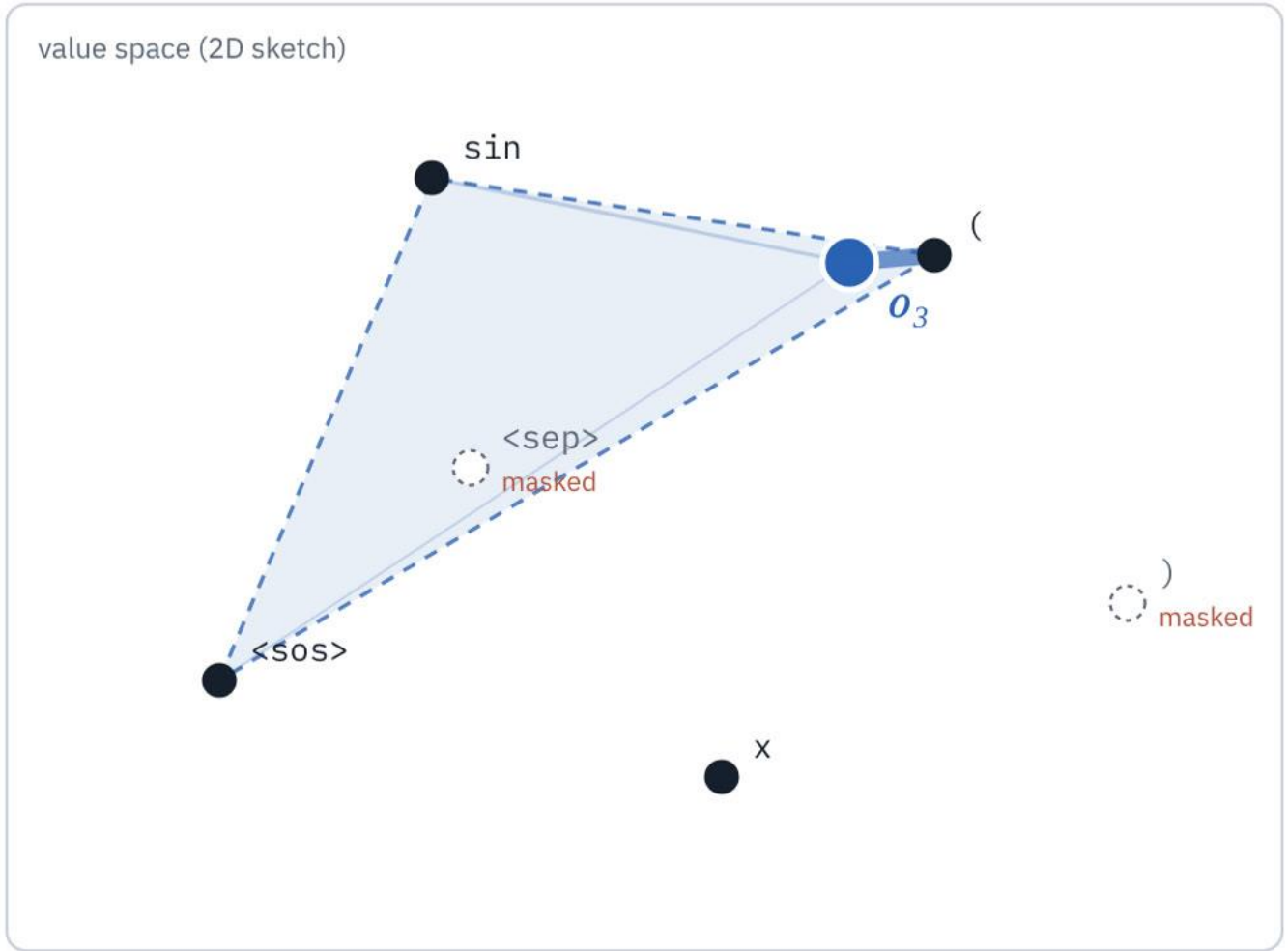
Σ = 1.00

$$o_k = \Sigma_{j \le k}\, w_{kj} \cdot V x_j$$

Query k = 2: the output is a weighted average of tokens 0…2; later tokens get weight 0.

Attention as a weighted average

one head · query token k



Weights for query k = 3 (x)

<sos>	0.04	
sin	0.11	
(	0.85	
x	0.00	
)	0 (masked)	
<sep>	0 (masked)	

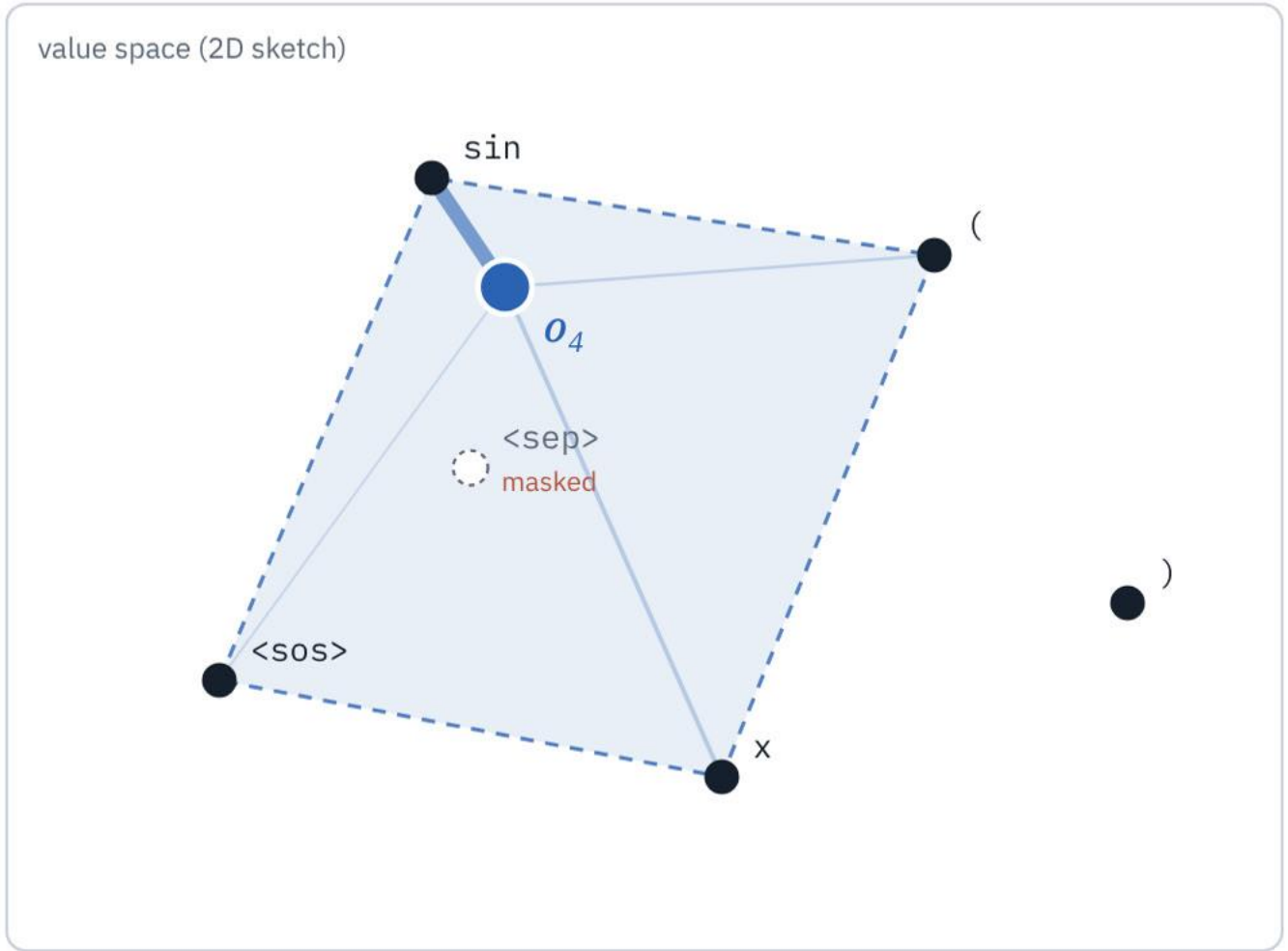
Σ = 1.00

$$o_k = \Sigma_{j \le k}\, w_{kj} \cdot V x_j$$

Query k = 3: the output is a weighted average of tokens 0…3; later tokens get weight 0.

Attention as a weighted average

one head · query token k



Weights for query k = 4 ())

<sos>	0.03
sin	0.75
(	0.07
x	0.15
)	0.00
<sep>	0 (masked)

Σ = 1.00

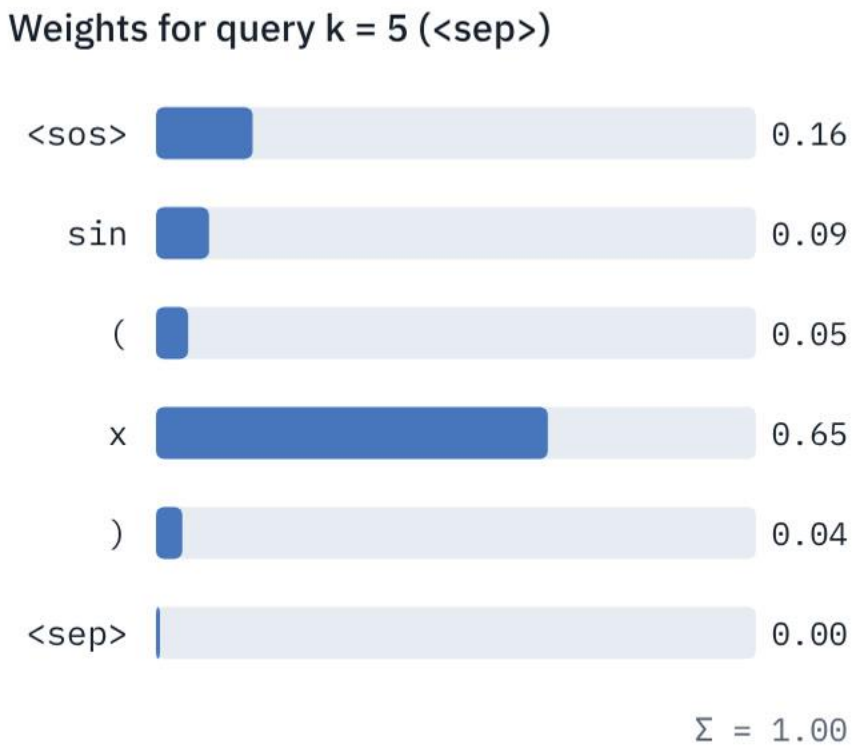
$$o_k = \Sigma_{j \le k}\, w_{kj} \cdot V x_j$$

Query k = 4: the output is a weighted average of tokens 0…4; later tokens get weight 0.

Attention as a weighted average

one head · query token k





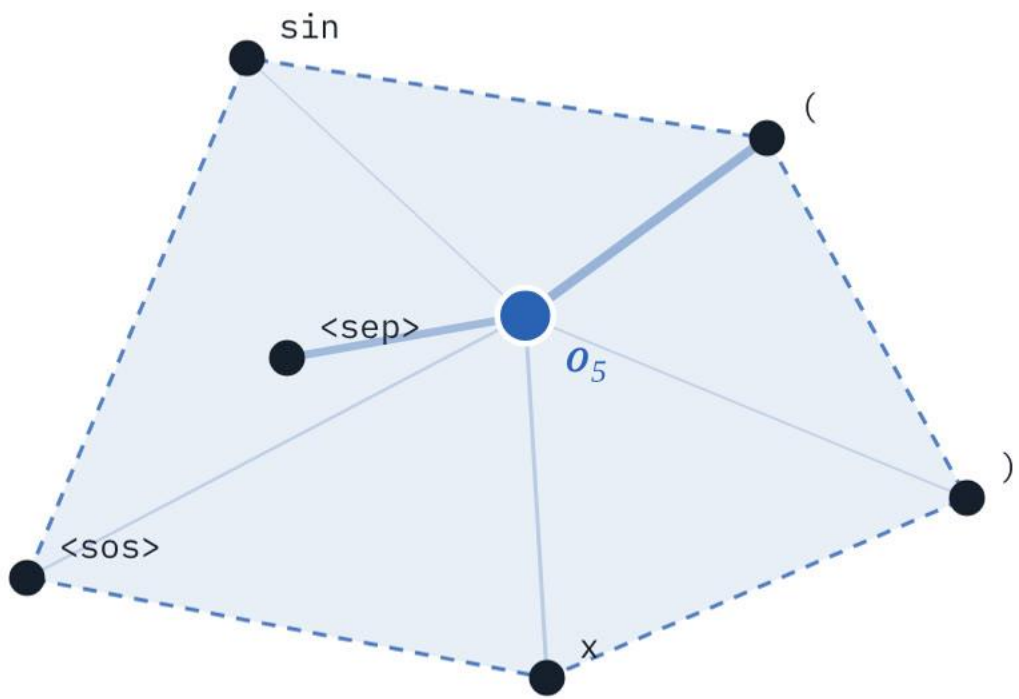
$$o_k = \Sigma_{j \le k}\, w_{kj} \cdot V x_j$$

Query k = 5: the output is a weighted average of tokens 0…5; later tokens get weight 0.

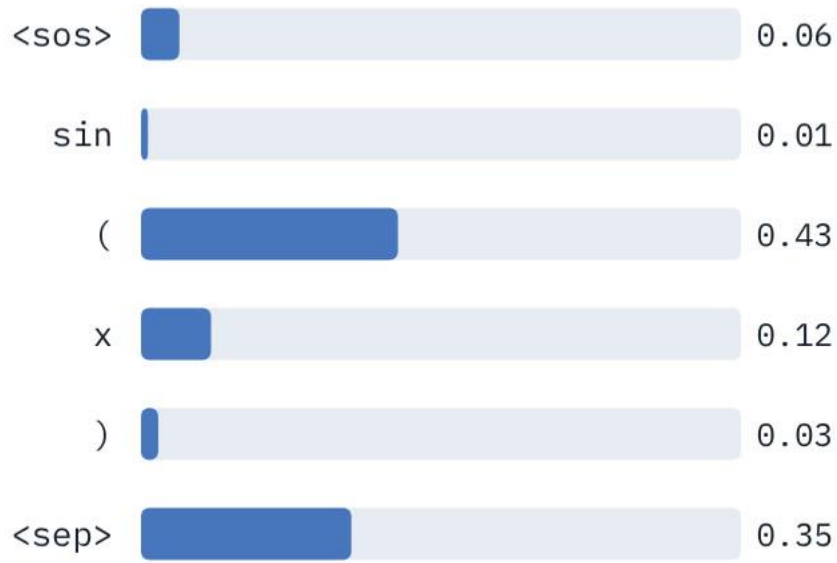
Attention as a weighted average

one head · query token k

value space (2D sketch)



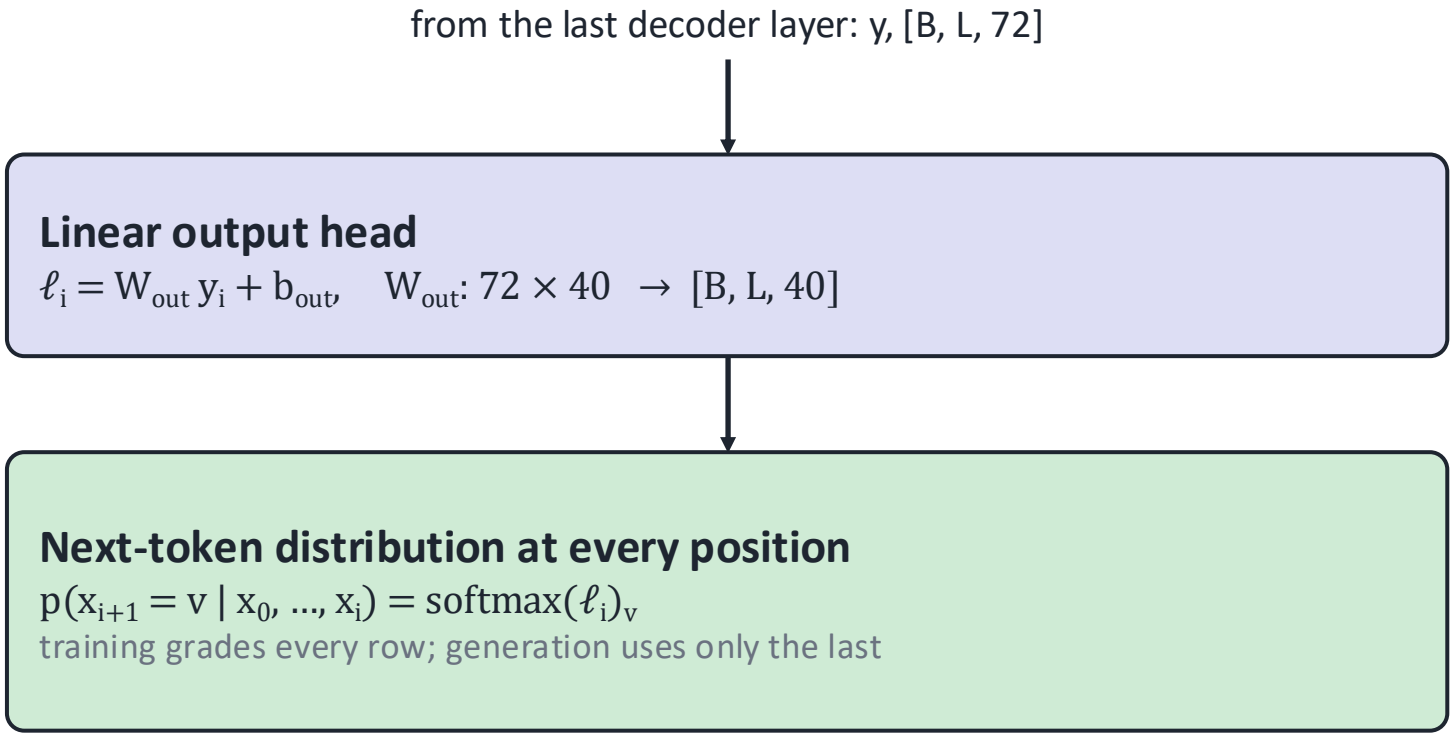
Weights for query k = 5 (<sep>)



Σ = 1.00

$$o_k = \Sigma_{j \le k}\, w_{kj} \cdot V x_j$$

TinyLM in detail 3/3: From vectors to next-token probabilities

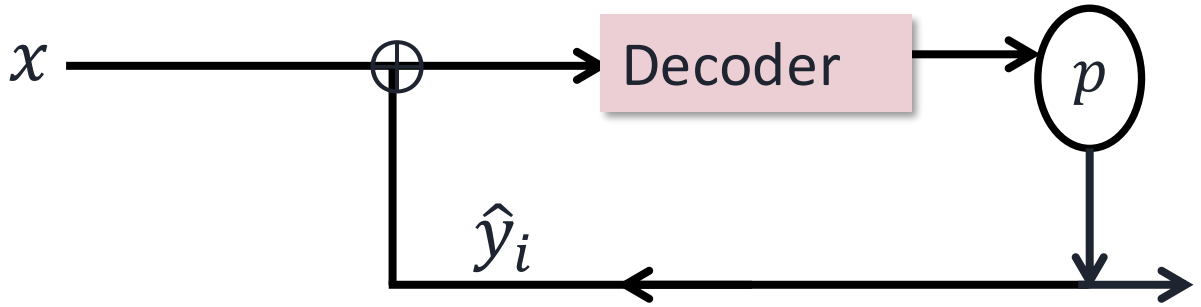


AUTOREGRESSIVE GENERATION

Autoregressive generation

Reminder:

- Append token $\langle sep \rangle$ to the prompt: $x \leftarrow [x, \langle sep \rangle]$.
- Decoder predicts probabilities for the next token for ***every*** sub-sequence; however, only the last sub-sequence, that is, the full sequence is relevant.
- Use probabilities of the last sub-sequence, to select the next output token, $\hat{y}_i$, which is appended to the current prompt x and the current output sequence $\hat{y}$.
- Repeat until the next token is either the $\langle eos \rangle$ or the token count limit is reached.



Autoregressive generation

greedy decoding · one token per forward pass

input: n = 6 tokens





rows 0…n−1

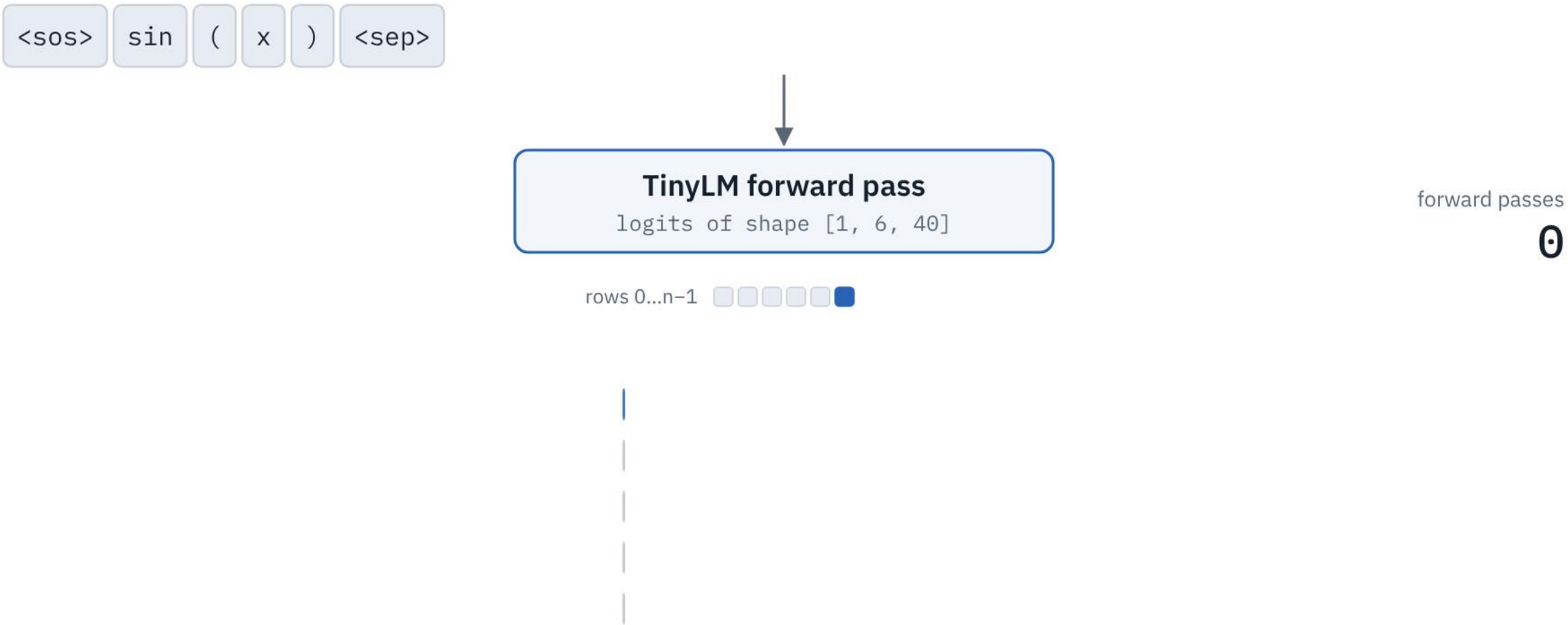
forward passes

0

Autoregressive generation

greedy decoding · one token per forward pass

input: n = 6 tokens



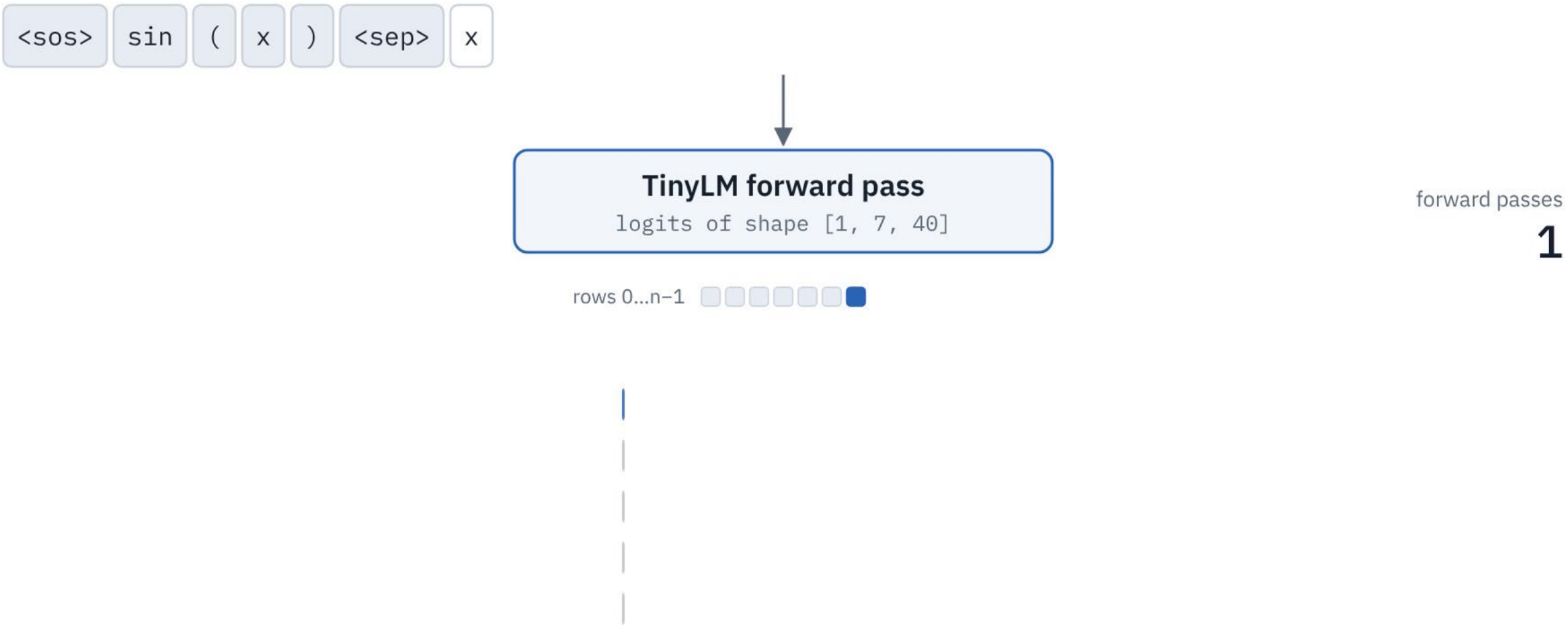
forward passes

0

Autoregressive generation

greedy decoding · one token per forward pass

input: n = 7 tokens



forward passes

1

Autoregressive generation

greedy decoding · one token per forward pass

input: n = 8 tokens



TinyLM forward pass

`logits of shape [1, 8, 40]`

rows 0…n−1

forward passes

2

Autoregressive generation

greedy decoding · one token per forward pass

input: n = 8 tokens





forward passes

3

rows 0…n−1 ← last row used

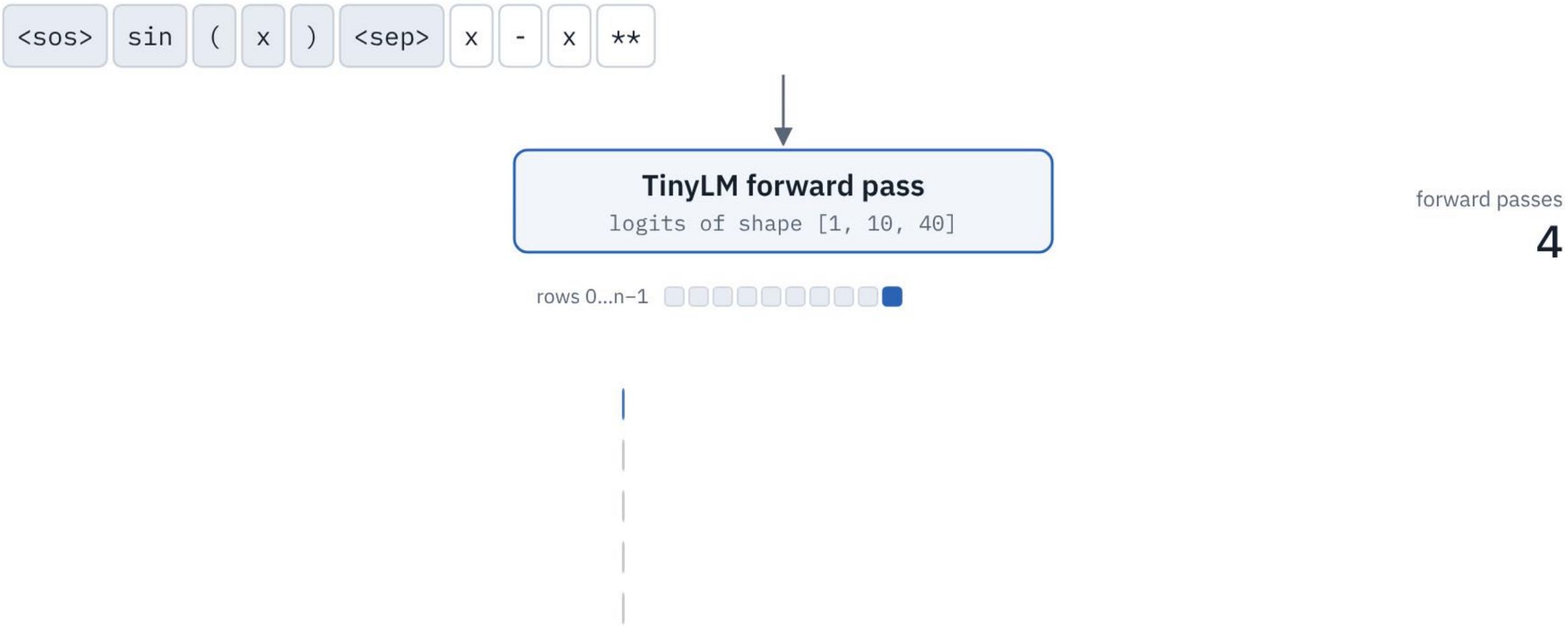
p(next token | all n tokens) illustrative



Autoregressive generation

greedy decoding · one token per forward pass

input: n = 10 tokens



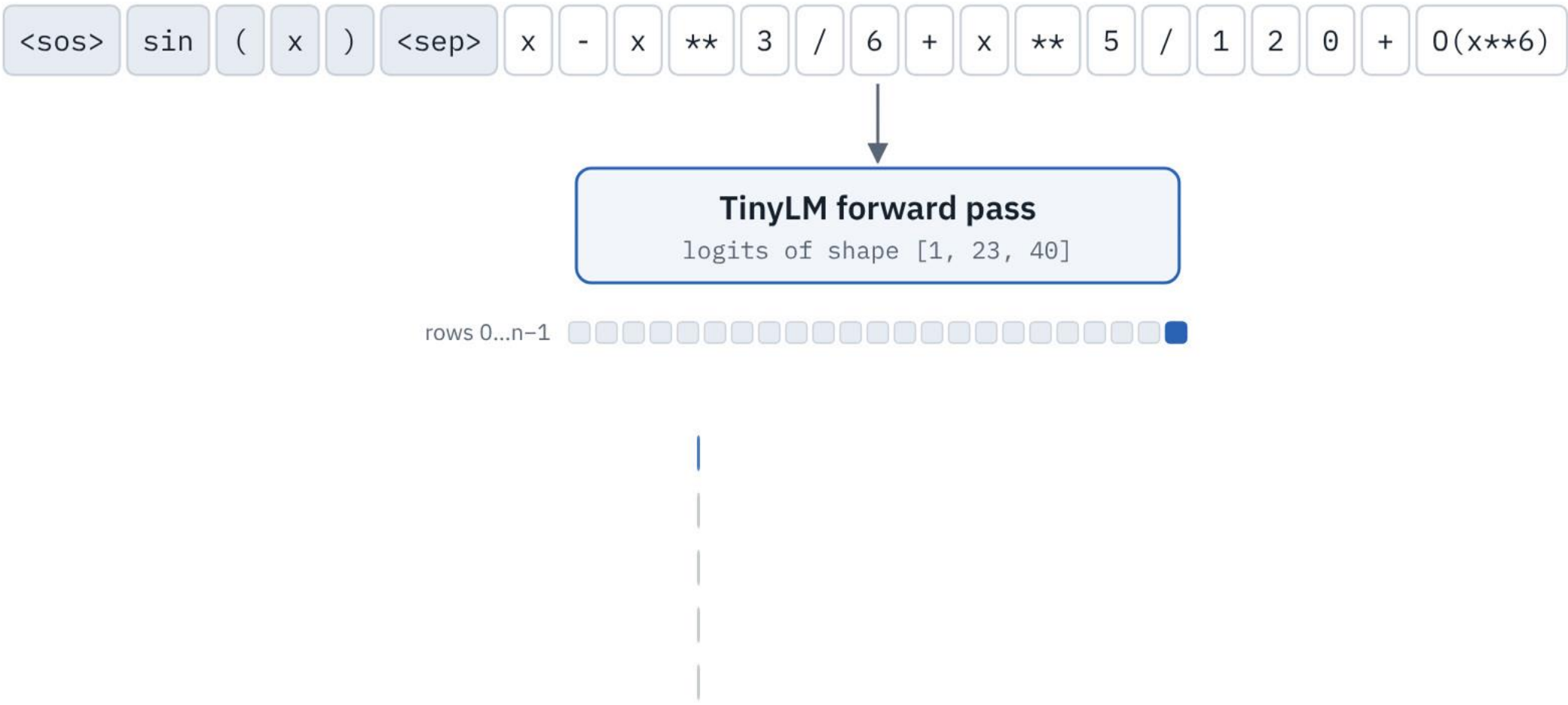
forward passes

4

Autoregressive generation

greedy decoding · one token per forward pass

input: n = 23 tokens



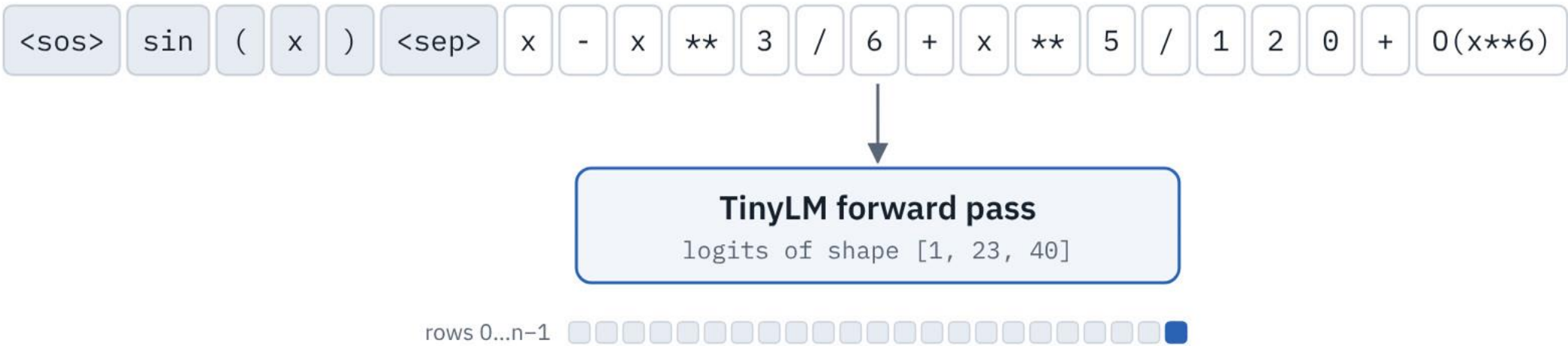
forward passes

18

Autoregressive generation

greedy decoding · one token per forward pass

input: n = 23 tokens



forward passes

18

SUMMARY

Summary

- Transformers based on the concept of attention have revolutionized the field of artificial intelligence.
- Variations of the model we have described underpins all current LLMs.
- But LLMs on their own have severe limitations, notably, their inability to use the vast quantity of precise information that is available to us. However, we shall see when we discuss AI agents, how the LLM revolution was super-charged by giving LLMs the ability to use tools, rather as the use of tools by our ancestors caused an exponential expansion in our species' ability to dominate the world (for better or worse).